UNITÀ 1 · CODIFICA DELLE INFORMAZIONI

Verifica Unità 1: soluzioni e griglia

Risposte corrette, punteggi e indicazioni di correzione. Totale 100 punti; voto in decimi = punti ÷ 10, arrotondato al mezzo voto (60 punti = 6).

1. Scelta multipla · 15 punti (2,5 ciascuna)

Dom.	Risposta corretta	Note
a	**programma e dati stanno nella stessa memoria**	Lezione "I sistemi di elaborazione".
b	**compilato in bytecode ed eseguito dalla JVM**	Lezione "Il software".
c	**il produttore conta 10^9 byte per GB, Windows divide per 1024^3**	$500 \cdot 10^9 / 1024^3 \approx 465{,}7$.
d	**−128**	Tranello: −0 esiste solo in modulo e segno.
e	**255**	parseInt legge "ff" in base 16 e restituisce un int. "11111111" sarebbe una stringa.
f	**7**	5 lettere ASCII + «é» da 2 byte (C3 A9). Più risposte segnate: 0 punti.

2. Vero o falso · 8 punti (1 ciascuna)

N.	V/F	Motivo
1	**F**	Il firmware è software di base: avvia l'hardware e carica il sistema operativo.
2	**F**	È il compilatore a tradurre tutto prima; l'interprete esegue un'istruzione alla volta.
3	**F**	I numeri romani sono un sistema non posizionale (salvo la regola della sottrazione).
4	**V**	$1 \cdot 8 + 7 = 15$.
5	**F**	Due zeri ci sono in modulo e segno e in complemento a 1; nel complemento a 2 lo zero è unico.
6	**V**	$0{,}5 = 2^{-1} = 0{,}1$ in binario.
7	**F**	Da 1 a 4 byte; i caratteri ASCII ne occupano 1.
8	**F**	L'int va in overflow in silenzio e diventa −2 147 483 648.

3. Conversioni · 12 punti (1,5 ciascuna)

Esercizio	Risultato	Esercizio	Risultato
10110110_2 → decimale	**182**	$110101110 1_2$ → esadecimale	**35D**
233_{10} → binario	**11101001**	200_{10} → esadecimale	**C8**

$3A7_{16}$ → decimale	**935**	1201_3 → decimale	**46**
755_8 → binario	**111101101**	$3E_{16}$ → ottale	**76**

Risultato giusto senza calcoli: 1 punto. Procedimento corretto con un errore di calcolo: 0,5 punti. Per 1101011101 → 35D l'errore tipico è raggruppare da sinistra (si ottiene D75): 0 punti per il risultato, 0,5 se il raggruppamento a 4 è comunque impostato. 3E → 76: si accetta anche il passaggio dal decimale (62 = 7 · 8 + 6).

4. Complemento a 2 · 15 punti

a) −45 = **1101 0011** (45 = 0010 1101, inverti 1101 0010, +1); −100 = **1001 1100** (100 = 0110 0100, inverti 1001 1011, +1). 2 punti ciascuno.

b) 1110 0110 = −128 + 64 + 32 + 4 + 2 = **−26**. 3 punti; 230 (lettura senza segno): 1 punto.

c) 0011 1100 + 1101 0011 = 1 0000 1111: il nono bit si scarta, risultato **0000 1111 = 15**. Codifica di −45 corretta 1 punto, somma con riporti 2 punti, verifica 1 punto. Chi parla di "overflow" per il nono bit perde 1 punto.

d) 0110 0100 + 0011 1100 = **1010 0000 = −96**; **c'è overflow**: due positivi danno un negativo, perché 160 > 127. Somma 2 punti, overflow riconosciuto 1 punto, motivazione 1 punto.

5. Le variabili nel debugger · 12 punti (2 ciascuna)

Variabile	Valore	Perché
m	**4.0**	9 / 2 è una divisione intera (4), poi diventa double.
n	**4.5**	Un operando è double: divisione reale.
t	**7**	Il cast (int) tronca, non arrotonda.
c	**'c'**	'a' = 97; 97 + 2 = 99 = 'c'.
ok	**false**	4.0 è diverso da 4.5.
x	**−126**	Restano 8 bit di 130 = 1000 0010, letti in complemento a 2: 130 − 256.

Valore giusto senza spiegazione: 1,5 punti. Accettato anche 4 per m solo se lo studente scrive che "in un double diventa 4.0". Per ok: se m è sbagliato, valutare la coerenza (errore portato: 1 punto).

6. IEEE 754 · 10 punti

a) 10,25 = 1010,01 = 1,01001 × 2^3. S = **1**; esponente 3 + 127 = 130 = **1000 0010**; mantissa **01001** seguita da 18 zeri; bit 1100 0001 0010 0100 0000 … = **C1240000**. Binario 2 punti, normalizzazione 1, segno 1, esponente 2, mantissa 1, esadecimale 1. Errore tipico: scrivere nella mantissa anche l'1 iniziale (−1 punto).

b) 0,1 in binario è **periodico** (0,000110011…): con un numero finito di bit va arrotondato. 2 punti; solo "è un numero con la virgola": 0 punti.

7. Caratteri · 10 punti

a) **Java** (2 punti). b) P = **50**, i = **69**, ù = **C3 B9**, spazio = **20**, € = **E2 82 AC**; **5 caratteri, 8 byte**. ù 2 punti, € 2 punti, gli altri byte e il conteggio 2 punti. Calcolo di ù: 0xF9 = 000 1111 1001 → 110 00011, 10 111001. c) **9volt, Zero, albero, zaino** (cifra 57 < Z 90 < a 97 < z 122); 2 punti, 1 se solo le maiuscole prima delle minuscole sono corrette.

8. Completa · 8 punti

program counter (PC) · **volatili** · **bytecode** · **3** · **r--** · **127** · **code point** · **breakpoint** (accettato "punto di interruzione"). Errori di ortografia che non cambiano il termine: nessuna penalità.

9. Domanda aperta · 10 punti

Risposta attesa: i bit non portano con sé il loro significato. 0100 0001 = 64 + 1 = **65** per una calcolatrice o un intero; la lettera **A** per un editor (ASCII/Unicode); un grigio scuro (intensità 65 su 255) per un'immagine. Il **dato** diventa **informazione** quando un programma o una persona lo interpreta con la **codifica** giusta. Un **modello linguistico** trasforma il testo in byte **UTF-8** e lo divide in **token** (pezzi di parola presi da un vocabolario), che poi elabora come numeri. Griglia: due interpretazioni con calcolo 4 punti; dato e informazione 2 punti; token 2 punti; uso corretto di almeno quattro termini 2 punti.

Documento riservato al docente. Per studenti con PDP o PEI: si possono dimezzare le voci degli esercizi 3 (conversioni) e 8 (completa) e riproporzionare il punteggio, oppure consentire la tabella delle potenze di 2 e la tabella ASCII.